

Relatorio Final do Compilador

1. Objetivo

O objetivo do projeto foi desenvolver um compilador didatico para uma linguagem simples, parecida com C, aplicando as principais etapas estudadas em teoria de linguagens formais e compiladores.

O compilador foi implementado em Python e cobre as seguintes fases:

1. analise lexica;
2. analise sintatica com geracao de AST;
3. analise semantica;
4. otimizacao simples;
5. geracao de codigo intermediario;
6. geracao de Assembly MIPS.

2. Linguagem Implementada

A linguagem aceita declaracoes, atribuicoes, estruturas condicionais, lacos e comandos simples de entrada e saida.

Exemplo:

```
int x;
bool positivo;

x = read();
positivo = x > 0;

if (positivo) {
    print(x);
} else {
    print(0);
}

while (x > 0) {
    x = x - 1;
}
```

Os tipos principais sao:

- 'int'
- 'bool'

Tambem existem literais de string, usados principalmente com print.

3. Analise Lexica

A analise lexica transforma o codigo fonte em uma sequencia de tokens.

O scanner reconhece:

- palavras reservadas;
- identificadores;
- numeros inteiros;
- strings;
- operadores aritmeticos;
- operadores relacionais;
- operadores logicos;
- delimitadores como parenteses, chaves e ponto e virgula;
- comentarios de linha com '//'.

Essa etapa esta implementada em:

compiler/lexer.py

Quando encontra um caractere invalido, o scanner informa linha e coluna do erro.

4. Analise Sintatica

A analise sintatica usa um parser descendente recursivo. A escolha desse metodo foi feita por ser mais simples de implementar e de explicar em um projeto didatico.

O parser verifica se a ordem dos tokens segue a gramatica da linguagem e monta uma AST.

Arquivos principais:

compiler/parser.py
compiler/ast_nodes.py

Exemplos de nos da AST:

- 'Program'
- 'VarDeclaration'
- 'Assignment'
- 'IfStatement'
- 'WhileStatement'
- 'PrintStatement'
- 'BinaryExpression'
- 'Literal'

5. Analise Semantica

A analise semantica verifica regras que nao dependem apenas da forma do programa.

Foram implementadas as seguintes verificacoes:

- variavel precisa ser declarada antes do uso;
- nao pode haver variavel repetida em escopos visiveis;
- nomes internos usados pelo compilador sao reservados;
- atribuicoes precisam respeitar o tipo declarado;
- condicoes de 'if' e 'while' precisam ser booleanas;
- operadores aritmeticos trabalham com 'int';
- operadores logicos trabalham com 'bool';
- 'read()' retorna 'int'.

Arquivo principal:

compiler/semantic.py

A tabela de simbolos foi implementada com uma pilha de escopos, permitindo tratar blocos criados com { e }.

6. Otimizacao

Foi implementada uma otimizacao simples antes da geracao de codigo intermediario.

As otimizacoes principais sao:

- simplificacao de expressoes constantes;
- simplificacao de 'if' com condicao constante;
- remocao de 'while (false)'.

Exemplo:

```
int x;  
x = 2 + 3 * 4;
```

Depois da otimizacao, a expressao passa a ser equivalente a:

```
x = 14;
```

Arquivo principal:

compiler/optimizer.py

7. Código Intermediário

Depois da análise semântica e da otimização, o compilador gera Código de Três Endereços.

Exemplo:

```
declare int x
t1 = read
x = t1
t2 = x > 0
ifFalse t2 goto L1
print x
L1:
```

Essa representação facilita a tradução para Assembly MIPS, pois divide expressões grandes em instruções menores.

Arquivo principal:

compiler/ir.py

8. Geração de Assembly MIPS

A etapa final traduz o código intermediário para Assembly MIPS.

O código gerado usa:

- '.data' para variáveis, temporários e strings;
- '.text' para instruções;
- 'syscall 5' para leitura de inteiro;
- 'syscall 1' para impressão de inteiro;
- 'syscall 4' para impressão de string;
- 'syscall 10' para encerrar o programa.

Arquivo principal:

compiler/mips.py

O Assembly pode ser impresso no terminal ou salvo em um arquivo .asm.

9. Como Executar

Executar o compilador imprimindo a saída no terminal:

```
python main.py exemplos/exemplo_basico.hc
```

Salvar o Assembly em arquivo:

```
python main.py exemplos/exemplo_basico.hc saida.asm
```

10. Testes

O projeto possui um script simples de testes:

```
python tests/run_tests.py
```

Os testes cobrem:

- programas validos;
- erro sintatico;
- erro semantico por variavel nao declarada;
- erro semantico por tipo incorreto;
- geracao de Assembly MIPS;
- criacao de arquivo '.asm';
- otimizacao de constantes.

Exemplos validos:

```
exemplos/exemplo_basico.hc  
exemplos/strings.hc  
exemplos/otimizacao.hc
```

Exemplos invalidos:

```
exemplos/erros/variavel_nao_declarada.hc  
exemplos/erros/tipo_incorreto.hc  
exemplos/erros/sintaxe_sem_ponto_virgula.hc
```

11. Limitacoes

Por ser um compilador didatico, algumas simplificacoes foram adotadas:

- nao existe funcao definida pelo usuario;
- nao existe vetor;
- nao existe tipo 'string' para variaveis;
- a geracao MIPS usa pseudo-instrucoes comuns em simuladores como MARS e SPIM;
- a otimizacao e simples e nao cobre todos os casos possiveis.

Mesmo com essas limitacoes, o projeto passa por todas as etapas principais de um compilador completo em escala reduzida.

12. Conclusao

O projeto implementa um fluxo completo de compilacao: o codigo fonte e lido, transformado em tokens, organizado em AST, verificado semanticamente, otimizado, convertido para codigo intermediario e traduzido para Assembly MIPS.

Com isso, o trabalho demonstra na pratica a ligacao entre os conceitos teoricos de linguagens formais, automatos, gramatica, semantica e geracao de codigo.